

SAM

An Agentic RAG Chatbot Built on LangGraph

A learning-ground-turned-product: conversational memory, grounded document Q&A, and live web search — orchestrated as one agent.

LangGraph

LangChain

GPT-4o-mini

FAISS

Streamlit

One agent, three capabilities users won't split apart.

Conversational continuity

Users expect follow-ups to remember earlier turns — not restart context on every message.

Grounded document Q&A

Answers about an uploaded PDF must cite the PDF, not the model's parametric memory.

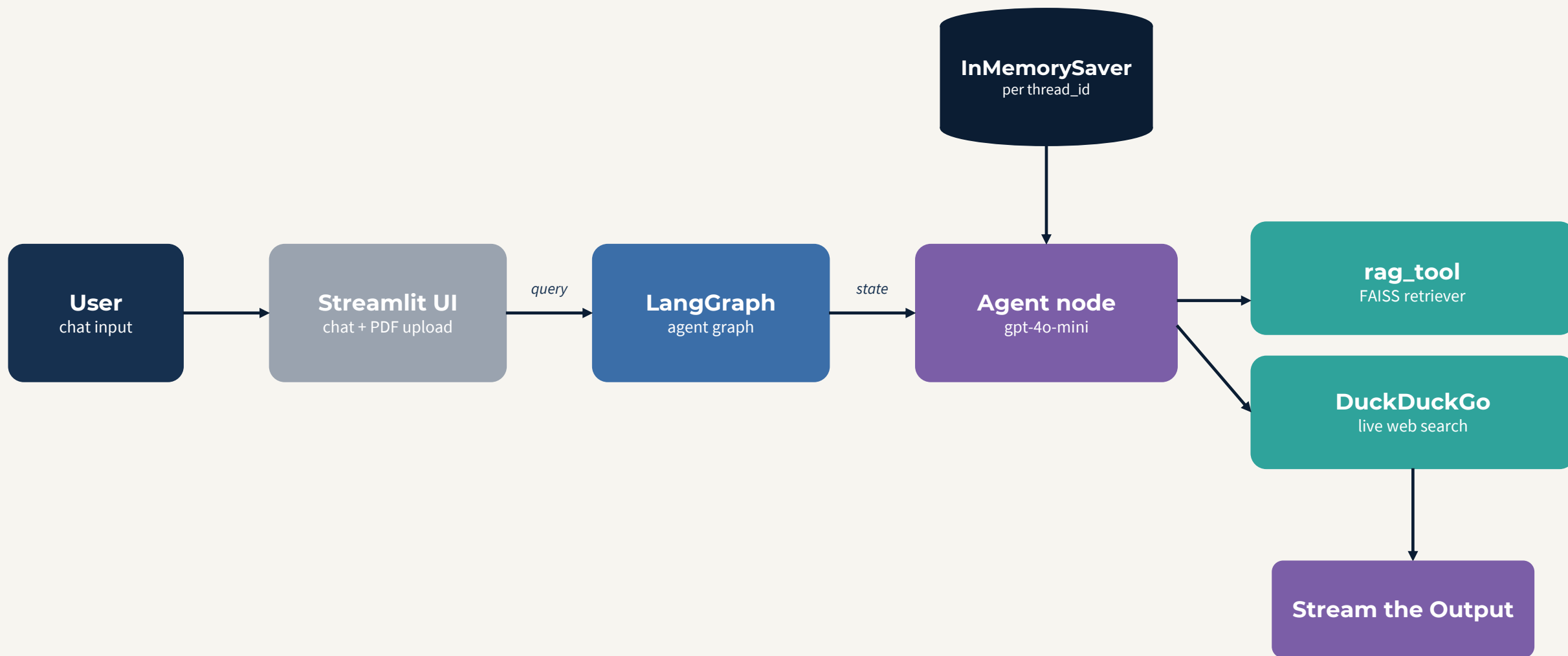
Live web knowledge

For anything time-sensitive, the agent must reach outside its training cutoff — on demand, not always.

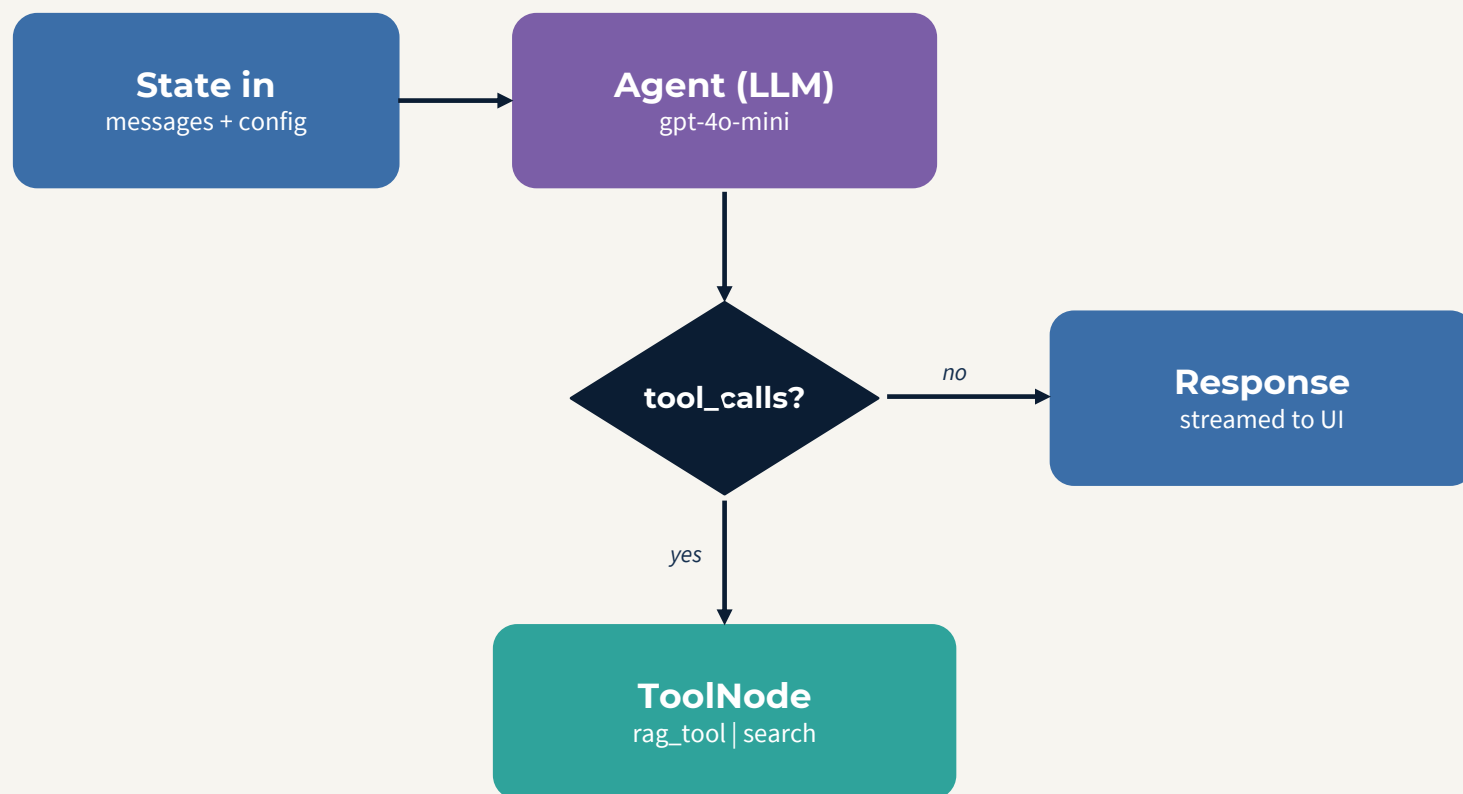
So what: without a single orchestrator that picks the right tool per turn, users end up juggling three interfaces — and hallucinations fill the gaps.



Request lifecycle, end to end.



The agent node decides: answer, or call a tool.



What actually persists

InMemorySaver checkpoints agent state keyed by `thread_id`.

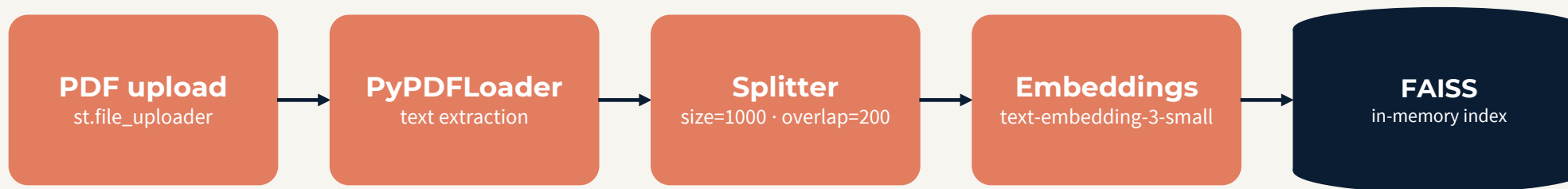
The extensibility seam

`config["configurable"]` passes per-invocation context into the graph — the same channel planned for location & timezone injection (see risk #3).

So what

The graph is agentic, not linear — tool selection happens per turn, and every future context signal has a defined entry point.

PDF → chunks → embeddings → per-thread FAISS.



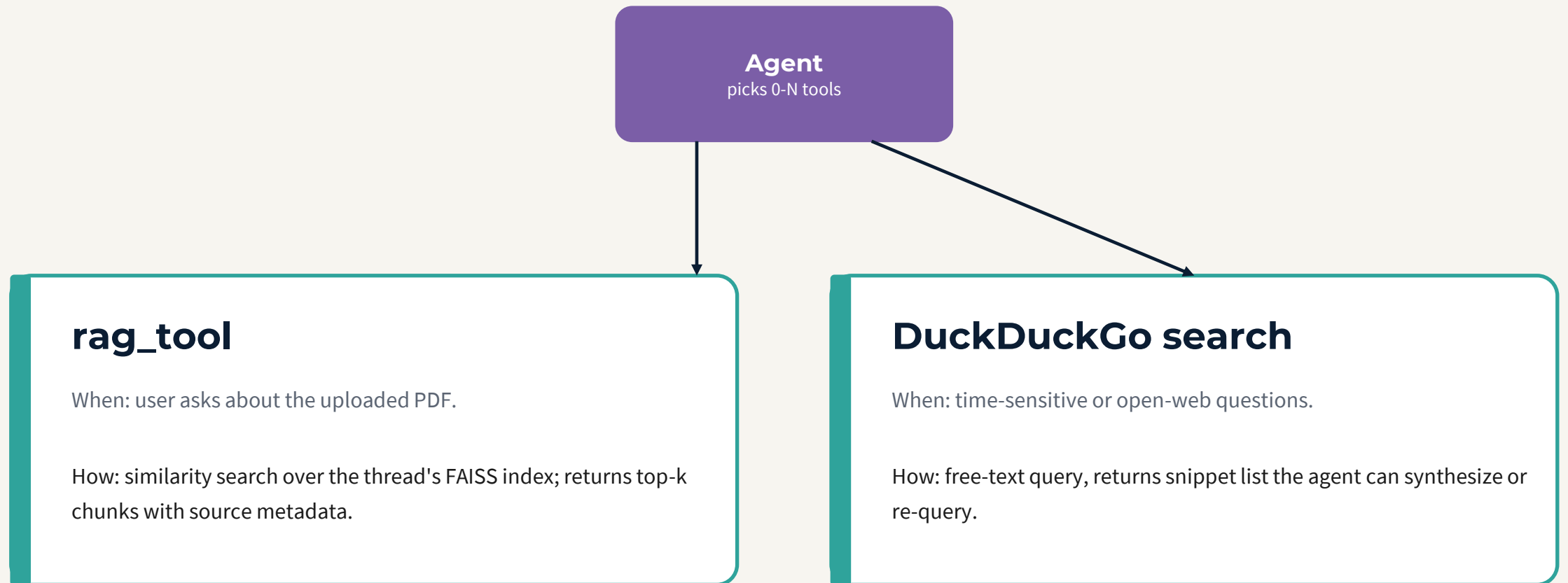
Per-thread isolation — the specific design decision

`_THREAD_RETRIEVERS` dict[thread_id → FAISS retriever]

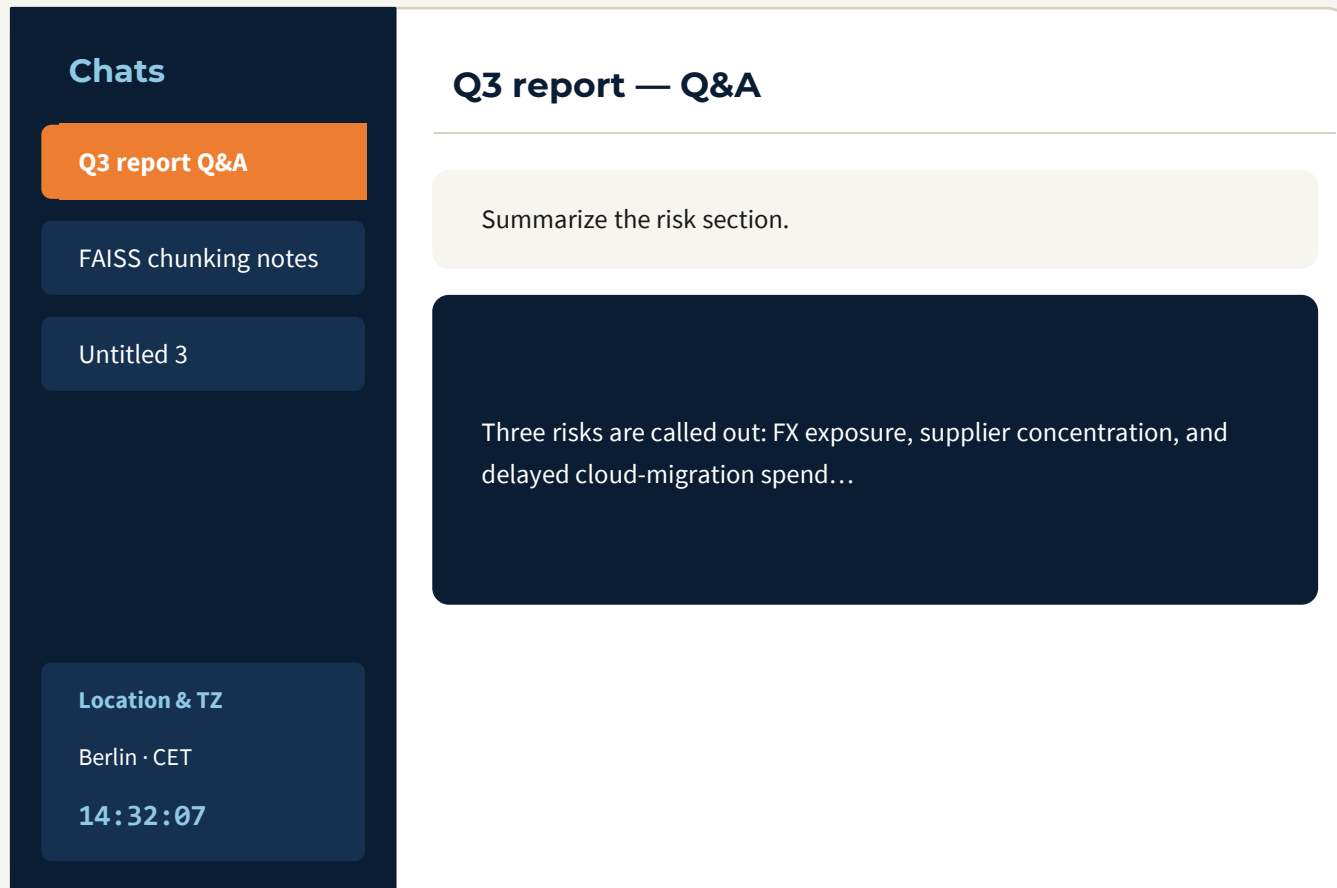
`_THREAD_METADATA` dict[thread_id → {filename, chunk_count, uploaded_at}]

Two module-level dicts scope every document upload to its own conversation — no cross-thread leakage, no shared index, no explicit tenancy layer to write.

Two peers. The agent chooses per turn.



Streamlit, but shaped deliberately.



Auto-generated, editable titles

First user turn seeds a title; user can rename inline.

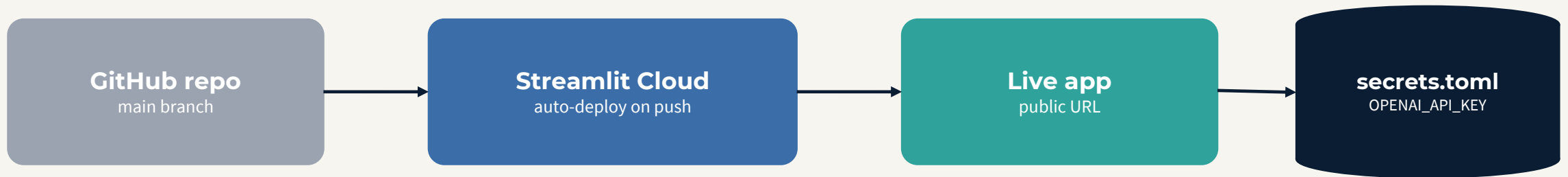
Collapsible Location & Time Zone panel

UI exists today; wiring into config is intentionally deferred (see risk #3).

Live clock via `st.fragment`

Only the clock re-renders — no full-app rerun cost per tick.

Streamlit Community Cloud. Secret in TOML. Done.



Framing: deployment simplicity is a feature at this stage — no infra to babysit, and the container-shaped work (persistent state, secrets rotation) is queued as an explicit next step, not an accident.

The system, in concrete values.

1000

chunk size (chars)

200

chunk overlap

1536

embedding dims • text-embedding-3-small

2

tools • rag_tool + DuckDuckGo

per-thread

FAISS + checkpointer scope

gpt-4o-mini

agent model

What's solid, what's deferred — named on purpose.

R1

Durable storage

RAG store and LangGraph checkpointer are in-memory. A restart wipes uploaded PDFs and conversation state.

High

R2

Unbounded agent loop cost

No hard cap on tool-call iterations per turn. Pathological queries can compound token spend before returning.

Medium

R3

UI/backend gap

Location & timezone panel is rendered but not yet piped into `config["configurable"]` — the agent doesn't see it.

Low

Now, next, later — anchored to the three risks.

NOW

- Persist FAISS index to disk (or swap for pgvector)
- Persist LangGraph checkpoint (SQLite → Postgres)
- *Addresses: R1*

NEXT

- Cap tool-call recursion + token budget per turn
- Emit per-turn cost/latency telemetry
- *Addresses: R2*

LATER

- Wire Location/TZ panel into `config["configurable"]`
- Add user auth so threads are user-scoped, not session-scoped
- *Addresses: R3*

Thanks.

Open to feedback and collaboration.

Stack

LangGraph · LangChain · gpt-4o-mini · text-embedding-3-small · FAISS · Streamlit

SAM